

**MORE ECMAScript**

## Javascript Server

Come accennato nel modulo EcmaScript 2015 anche a lato server sono state introdotte nuove funzionalità riguardo la programmazione asincrona.

```
async function fetchAPI(){  
  const Url= 'https://jsonplaceholder.typicode.com/todos'  
  const response = await fetch(Url)  
  const resp = await response.json();  
  return resp;  
}  
  
fetchAPI().then(console.log);
```

In questo esempio viene definita una funzione asincrona(`async`) richiedendo dei dati a Url attraverso il metodo `fetch()`(ottenere) `await`(attesa di risposta dal server), ritornerà una `resp` con dati in json da parte di Url che sarà il server a cui stiamo richiedendo i dati, `.then` è una promise in cui tornerà lo status della richiesta dei nostri dati se verrà risolta o rigettata.

**async await** = funzione asincrona

**fetch()**= metodo per ottenere dati dal server

**.then**= ritorna lo stato di risposta del server

## Introduzione ai Json

I json sono oggetti definiti da proprietà e valore racchiusi tra parentesi graffe utilizzati per l'archiviazione e lo scambio di dati.

Esempio di dati json:

```
'{"name":"John", "age":31, "city":"New York"}'.
```

### Regole di sintassi JSON

La sintassi JSON deriva dalla sintassi della notazione degli oggetti JavaScript:

- I dati sono in coppie nome / valore
- I dati sono separati da virgole
- Le parentesi graffe ricci tengono gli oggetti
- Le parentesi quadre contengono gli array

### Valori JSON

In **JSON** , i *valori* devono essere uno dei seguenti tipi di dati:

- una stringa
- un numero
- un oggetto (oggetto JSON)
- un array
- un booleano
- nullo

Accedere al Json con un oggetto Javascript:

```
<!DOCTYPE html>
<html>
<body>
<p id="demo"></p>
```

```
<script>
var myObj, obj;
myObj = { name: "John", age: 30, city: "New York" };
```

```
obj= myObj.name;  
document.getElementById("demo").innerHTML = obj;  
</script>  
  
</body>  
</html>
```

Conversione di dati json in oggetto Javascript

```
<html>  
<body>  
  
<h2>Convert a string written in JSON format, into a JavaScript object.</h2>  
  
<p id="demo"></p>  
  
<script>  
var myJSON = '{"name":"John", "age":31, "city":"New York"}';  
var myObj = JSON.parse(myJSON);  
document.getElementById("demo").innerHTML = myObj.name;  
</script>  
  
</body>  
</html>
```

Quando si inviano dati a un server Web, i dati devono essere una stringa.

Conversione di un oggetto JavaScript in una stringa con JSON.stringify().

```
let obj = { name: "John", age: 30, city: "New York" };  
let myJson = JSON.stringify(obj);
```

Array negli oggetti JSON

Gli array possono essere valori di una proprietà di un oggetto:

Esempio

```
{  
  "name": "John",  
  "age": 30,  
  "cars": [ "Ford", "BMW", "Fiat" ]  
}
```

## Introduzione a Node JS

Node JS è una tecnologia lato server scritta in Javascript che permette di occuparsi di tutta la tecnologia backend di un'applicazione, comprende diversi moduli che riguardano sia il fileSystem per leggere o scrivere su un file ad es. Word, pdf, txt. Permette di creare un web server e di gestire il protocollo di rete http request e response.

## Creare FileSystem

```
const fs = require('fs');  
fs.writeFileSync('hello.txt', 'ciao benvenuti a tutti al corso di Node Js ');  
  
fs.writeFileSync('hello.pdf', 'sono un PDF');
```

## Creare Web Server

```
let http = require('http');
let server = http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.write('Hello World');
  res.end();
});
server.listen(1337, '127.0.0.1');
console.log('Server running at http://127.0.0.1:1337/');
```

## Introduzione ad Express JS

ExpressJS è un framework per applicazioni Web che fornisce una semplice API per creare siti Web, app Web e back-end.

Con ExpressJS, non devi preoccuparti di protocolli, processi, ecc.

Express fornisce un'interfaccia minima per creare le nostre applicazioni. Ci fornisce gli strumenti necessari per creare la nostra app. È flessibile in quanto sono disponibili numerosi moduli su **npm**, che possono essere collegati direttamente a Express.

Express è stato sviluppato da **TJ Holowaychuk** ed è gestito dalla fondazione Node.js e da numerosi collaboratori open source.

A differenza dei suoi concorrenti come Rails e Django, che hanno un modo supponente di costruire applicazioni, Express non ha il "modo migliore" per fare qualcosa. È molto flessibile e collegabile.

## Creazione di un app con Express JS

```
Let express = require('express');var app = express();app.get('/', function(req, res){
  res.send("Hello world!");});app.listen(3000);
```

Localhost:3000//

indirizzo

url

Hello Word// output dell'app

## Creazione di app Router

```
Let express = require('express');var router = express.Router();router.get('/',  
function(req, res){ res.send('GET route on things.')});router.post('/', function(req,  
res){ res.send('POST route on things.')});//export this router to use in our  
index.jsmodule.exports = router;
```

Ora per utilizzare questo router nel file **index.js** , digitiamo quanto segue prima della chiamata alla funzione **app.listen** .

```
let express = require('Express');var app = express();  
let things= require('./things.js');  
app.use('/things', things);app.listen(3000);
```

## Creazione di un URL

```
Let express = require('express');var app = express();app.get('/:id', function(req, res){  
res.send('The id you specified is ' + req.params.id)});  
app.listen(3000);
```

Localhost:3000/123 // indirizzo URL

The id you specified is 123 //output app